



@ADM126

JUNE 22-26, 2026

The Hacking Tribunal!

Future Event Prediction



Team Introduction



Jonathan Wembolua
Graph Analytics & Prediction Lead
Computer Science, *WSSU*



Madison Conley
AI Research & Insight Lead
Computer Science, *MVSU*



Nadia Rapheal
Technical Program Manager & Integration Lead
Computer Science, *WSSU*



Josiah Johnson
Frontend, Visualization & Presentation Lead
Computer Science, *WSSU*



Usir Jackson
Data & Knowledge Graph Lead
Computer Science, *Claflin*

Usir Jackson

Data & Knowledge Graph Lead

Computer Science, Claflin



Milestones:

Biological Seed Dataset Creation

1. Select initial biological seed articles.
2. Extract Wikipedia article links and categories.
3. Generate structured JSON datasets.

Knowledge Graph Construction

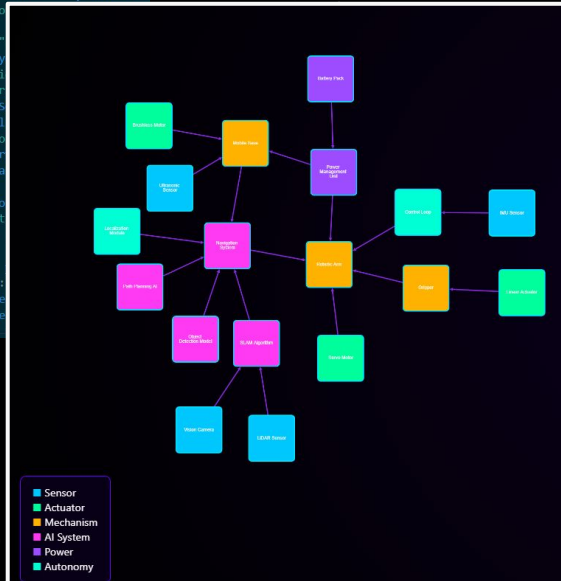
1. Create nodes from biological concepts.
2. Create relationships between connected concepts.
3. Generate graph visualization output.

```

index.html X  robotics_knowledge_base_with_predictions.json X
index.html > html > body > script > LINKS
2 <html lang="en">
46 <body>
66 <script>
67 const NODES = [
68   { id: "Robotics", label: "Robotics", cluster: "Core", yearSort: 1952, node_type: "category",
69
70   { id: "Acceleration", label: "Acceleration", cluster: "Foundation", yearSort: 1980, node_type: "concept",
71   { id: "Analytical mechanics", label: "Analytical mechanics", cluster: "Foundation", yearSort: 1980, node_type: "concept",
72   { id: "Appell's equation of motion", label: "Appell's equation of motion", cluster: "Foundation", yearSort: 1980, node_type: "concept",
73   { id: "Angular acceleration", label: "Angular acceleration", cluster: "Foundation", yearSort: 1980, node_type: "concept",
74   { id: "Angular displacement", label: "Angular displacement", cluster: "Foundation", yearSort: 1980, node_type: "concept",
75   { id: "Angular momentum", label: "Angular momentum", cluster: "Foundation", yearSort: 1980, node_type: "concept",
76   { id: "Angular velocity", label: "Angular velocity", cluster: "Foundation", yearSort: 1980, node_type: "concept",
77
78   { id: "Automatic control", label: "Automatic control", cluster: "Foundation", yearSort: 1980, node_type: "concept",
79   { id: "Kinematics", label: "Kinematics", cluster: "Foundation", yearSort: 1980, node_type: "concept",
80   { id: "Manufacturing", label: "Manufacturing", cluster: "Foundation", yearSort: 1980, node_type: "concept",
81   { id: "Manufacturing Engineering", label: "Manufacturing Engineering", cluster: "Foundation", yearSort: 1980, node_type: "concept",
82   { id: "Actuator", label: "Actuator", cluster: "Foundation", yearSort: 1980, node_type: "concept",
83   { id: "Industrial automation", label: "Industrial automation", cluster: "Foundation", yearSort: 1980, node_type: "concept",
84   { id: "Industrial robot", label: "Industrial robot", cluster: "Foundation", yearSort: 1980, node_type: "concept",
85   { id: "Aerobot", label: "Aerobot", cluster: "Foundation", yearSort: 1980, node_type: "concept",
86   { id: "AI winter", label: "AI winter", cluster: "Foundation", yearSort: 1980, node_type: "concept",
87
88   { id: "ABB", label: "ABB", cluster: "Industrial Robotics", yearSort: 1980, node_type: "entity",
89   { id: "Robotic sensors", label: "Robotic sensors", cluster: "Control", yearSort: 1980, node_type: "entity",
90   { id: "Accelerometer", label: "Accelerometer", cluster: "Control", yearSort: 1980, node_type: "entity",
91
92   { id: "ASIMO", label: "ASIMO", cluster: "Humanoid and Embodied", yearSort: 1980, node_type: "entity",
93   { id: "Adaptable robotics", label: "Adaptable robotics", cluster: "Present", yearSort: 1980, node_type: "entity",
94   { id: "3D scanning", label: "3D scanning", cluster: "Present", yearSort: 1980, node_type: "entity",
95   { id: "3D printing", label: "3D printing", cluster: "Present", yearSort: 1980, node_type: "entity",
96   { id: "Ballbot", label: "Ballbot", cluster: "Present", yearSort: 1980, node_type: "entity",

```

< json nodes for the graph visualization



^ interface

Programming Languages:

- JavaScript
- Python

Libraries and Frameworks:

- React
- Cytoscape and Cytoscape-fcose
- React Router

Development Tools:

- VS Code
- Git
- GitHub

Platforms:

- Google Colab
- GitHub Pages

AI Resources:

- Copilot (code assistance, debugging)
- Claude (interface development, and implementation support)

Madison Conley

AI Research & Insight Lead

Computer Science, MVSU



Milestones: AI Research & Insight

- Generate AI predictions for robotics concepts.
- Analyze historical predecessors of emerging technologies.
- Forecast future technology trajectories using an LLM.
- Evaluate technological impact across related fields.
- Enrich the knowledge base with AI-generated insights.
- Produce explainable predictions grounded in historical data.

```

msg = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=1000,
    messages=[{"role": "user", "content": pred_prompt}]
)

raw = msg.content[0].text.strip()

# DEBUG: Print what we got back
print(f" [DEBUG] Predecessors response length: {len(raw)}")
if len(raw) > 0:
    print(f" [DEBUG] First 100 chars: {raw[:100]}")
else:
    print(f" [DEBUG] EMPTY RESPONSE")
    return None

predictions["predecessors"] = clean_and_parse_json(raw)
print(f" ✓ Predecessors analyzed")
time.sleep(1)

```

v predictions and context

```

def predict_node(node, G, kb, analysis_type="all", curated_set=None):
    """
    Generate predictions for a single node.
    analysis_type: "predecessors", "forward", "backward", or "all"
    curated_set: Set of curated forecast concepts (optional)
    """
    predictions = {}

    # Skip if node not in graph (no relationships)
    if node not in G:
        print(f" ⚠ Node not in graph (no relationships) – skipping")
        return None

    # Get context
    predecessors = get_predecessors(node, G, depth=2)
    successors = get_successors(node, G, depth=2)
    context = get_node_context(node, kb, G)

    # Add priority flag if in curated set
    if curated_set and node.lower() in curated_set:
        predictions["is_curated_forecast_priority"] = True

    try:
        # PREDECESSORS ANALYSIS
        if analysis_type in ["predecessors", "all"]:
            pred_prompt = PREDECESSOR_PROMPT.format(
                node=node,

```

< creating calls to anthropic

```

# prediction prompt templates

PREDECESSOR_PROMPT = """\
You are a robotics historian analyzing concept evolution.

Given the emerging robotics concept: "{node}"

Historical predecessors that likely led to it: {predecessors}

Based on robotics history, what key technologies or concepts from the past
enabled or directly led to "{node}"? Be specific about HOW they connect.

Return ONLY valid JSON (no markdown, no preamble):

{{
  "direct_predecessors": ["concept 1", "concept 2"],
  "enabling_technology": "description of how they enabled this",
  "timeline_origin": "approximate decade when these predecessors were relevant"
}}
"""

```

^ prediction prompt

Programming Language:

Python

Libraries and Frameworks:

- Anthropic
- NetworkX
- JSON
- time (API rate limiting)
- collections (batch processing)

Data Produced:

- AI Generated historical predecessor predictions
- Enriched robotics knowledge base

Platforms:

- Google Colab
- Anthropic API

AI Resources:

- Anthropic Claude Haiku (LLM for historical analysis and future prediction)

Nadia Rapheal

Technical Program Manager & Integration Lead

Computer Science, WSSU



Technical Program Manager & Integration
Lead

Coordinate project milestones and team
deliverables.

Integrate project components into a unified
pipeline.

Validate data flow across all system modules.

Manage testing, debugging, and system
integration.

Maintain project documentation and
technical reports.

Prepare the final project for demonstration
and delivery.

THE HACKING TRIBUNAL

5 - DAY HACKATHON GANTT CHART

Duration: 5 Days

Monday → Friday

TEAM: Nadia, Josiah, Usir, Jonathan, Madison

	ID	Task	Owner	Monday - 1	Tuesday - 2	Wednesday - 3	Thursday - 4	Friday - 5
1	1	Repo Setup + README init	Josiah, Nadia					
	2	Colab + API Config System	Nadia					
2	3	Robotics Seed Article Selection	Usir					
	4	WikiAPI Recursive Scraper	Usir					
	5	Token Parser: Concept-Relation	Nadia					
3	6	Knowledge Graph Node Schema Design	Usir					
	7	Edge Construction + Wiki Link Mapping	Usir					
	8	NetworkX Graph Storage + Serialization	Josiah					
4	9	Historical Snapshot Pipeline	Jonathan					
	10	Interactive Graph UI Base	Josiah					
5	11	LLM Engine Init	Madison					
	12	Temporal Evolution Analysis	Jonathan					
	13	Centrality Metrics Computation	Jonathan					
	14	Convergence Detection System	Nadia					
6	15	RAG Forecasting Engine	Madison					
	16	Trend Prediction Output	Madison					
7	17	UI Creation and Design	Josiah					
	18	UI Expansion (Time, Cluster, Score)	Josiah					
8	19	Dashboard and Website	Josiah, Nadia					
	20	Final README + Poster + Demo Prep	ALL TEAM					

Jonathan Wembolua

Graph Analytics & Prediction Lead

Computer Science, WSSU



Milestones:

Historical Evolution Analysis

- Analyze historical growth trends across robotics technologies.
- Calculate forecasting metrics from graph and temporal data.
- Rank emerging concepts by forecast score and confidence.

Convergence Detection System

- Detect technologies converging across robotics subfields.
- Analyze graph connectivity to identify innovation hotspots.
- Generate prediction datasets for future trend analysis.

```

# Calculate component scores
growth = year_growth_score(years_list)
recency = recency_score(years_list)
connectivity = connectivity_score(node, G)
subfield = emerging_subfield_score(node, kb, article_by_concept)

# PREDICTION FORMULA
# Emerging concept = high growth + high recency + low connectivity + high subfield presence
# Weight: growth and subfield are strongest signals
forecast_score = (
    (growth * 0.35) +          # rapid year mention growth
    (subfield * 0.35) +        # appearing in multiple subfields
    (recency * 0.20) +         # high recency
    ((1 - connectivity) * 0.10) # not yet fully integrated
)

```

```

for pred in predictions:
    print(f"\n#{pred['rank']} - {pred['concept'].upper()}")
    print(f"  Confidence: {pred['confidence']}")
    print(f"  Timeline: {pred['timeline']}")
    print(f"  Forecast Score: {pred['forecast_score']}")
    print(f"  Summary: {pred['summary']}")

print("\n" + "=" * 80)
print("Ready for dashboard integration!")

```

```

=====
ROBOTICS FORECASTING REPORT - 2026-2036
=====

```

```

#1 - AUTONOMOUS AGRICULTURAL VEHICLES
Confidence: High
Timeline: 2026-2036
Forecast Score: 0.993
Summary: Autonomous agricultural vehicles shows rapidly growing ment

```

^ prediction formula

< forecasting report output

Programming Language:

Python

Libraries:

- json – Loading and processing the robotics knowledge base
- networkx – Knowledge graph construction, graph traversal, and network analysis
- collections – Efficient aggregation of concept statistics and relationships
- statistics – Calculation of historical trend metrics and averages
- cytoscape – Compatibility with the visualizer

Forecasting Methods:

- Historical year growth analysis
- Recency scoring
- Graph connectivity analysis
- Confidence score generation
- Forecast ranking and prediction filtering

Platforms:

- Google Colab

Josiah Johnson

Frontend, Visualization & Presentation Lead

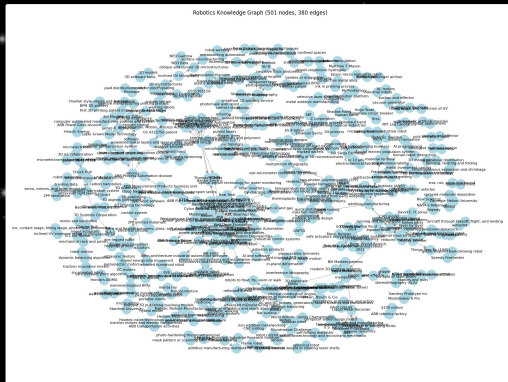
Computer Science, WSSU



Milestones:

Visualization and User Experience

- Develop the interactive robotics knowledge graph interface.
- Visualize concept relationships and prediction results.
- Integrate forecasting data into dashboard components.
- Design presentation graphics and project visuals.
- Organize project documentation and deliverables.
- Demonstrate project functionality during presentations.



v getting article text and links

```
# wikipedia api helpers
def get_article_text(title: str) -> str:
    """Fetch the plain-text extract of a Wikipedia article."""
    url = (
        "https://en.wikipedia.org/w/api.php"
        "?action=query"
        "&prop=extracts"
        "&explaintext=1"
        "&format=json"
        f"&titles={requests.utils.quote(title)}"
    )
    try:
        r = requests.get(url, headers=HEADERS, timeout=15)
        r.raise_for_status()
        page = next(iter(r.json()["query"]["pages"].values()))
        return page.get("extract", "")
    except Exception as e:
        print(f" ⚠ Text fetch failed for '{title}': {e}")
        return ""

def get_article_links(title: str) -> list:
    """Return Wikipedia article titles linked from the given article (main namespace only)."""
    url = (
        "https://en.wikipedia.org/w/api.php"
        "?action=query"
        "&prop=links"
        "&pllimit=max"
    )
```

< visualization output

```
C: > Users > nadia > Downloads > LARGE_knowledge_base.json > ...
1 {
2   "articles": {
3     "Robotics": { ...
51   },
52   "IX Technologies": { ...
77   },
78   "ABB": { ...
168  },
169   "ASIMO": { ...
220  },
221   "Adaptable robotics": { ...
241  },
242   "Aerobot": { ...
274  },
275   "Agibot": { ...
305  },
306   "Agility Robotics": { ...
335  },
336   "Agricultural robot": { ...
362  },
363   "Al-Da": { ...
395  },
396   "Amazon Robotics": {
397     "title": "Amazon Robotics",
398     "depth": 2,
399     "parent": "IX Technologies",
400     "text_length": 4005,
401     "categories": [
402       "All articles needing rewrite",
403       "All articles with a promotional tone",
404       "All pages needing factual verification",
405       "Amazon (company) acquisitions",
406       "Articles with a promotional tone from December 2014",
407       "Articles with multiple maintenance issues",
408       "Articles with short description",
409       "CS1 German-language sources (de-es)"
    ]
  }
}
```

^ result of wiki crawl

AI Resources:

Anthropic Claude Sonnet 4.6

Libraries:

- requests – Wikipedia API requests
- json – Knowledge base serialization
- time – Rate limiting during crawling
- re – Historical year extraction
- anthropic – Claude API integration
- networkx – Knowledge graph construction and graph analysis

Data Sources:

- Wikipedia API
- Internal article links
- Article categories

Platforms:

- Google Colab
- Anthropic API
- Wikipedia API